

PROJECT ETHER

Implementation Pack v4.0

Status: Implementation Ready

Base: Blueprint v3.0

Objetivo: transformar o design do Project Ether em uma especificação executável por agentes de desenvolvimento.

1. PRINCÍPIO FUNDAMENTAL

O agente de desenvolvimento deve considerar este documento como contrato.

Ele deve:

- implementar;
- testar;
- documentar;
- validar;
- reportar gaps.

Ele **não deve inventar regras de produto.**

Quando encontrar algo não definido:

SPEC_GAP

Quando encontrar duas regras incompatíveis:

SPEC_CONFLICT

Quando precisar de decisão de produto:

PRODUCT_DECISION_REQUIRED

2. STACK OFICIAL DO MVP

Backend

.NET		10
C#		14
ASP.NET		Core
Entity Framework	Framework	Core
PostgreSQL		
Redis		
WebSockets		
OpenTelemetry		

Client

Godot		4.x
GDScript	ou	C#
2D		
Android		
iOS		

Infrastructure

Linux		
Docker		
Docker		Compose
GitHub		Actions

3. DECISÃO IMPORTANTE — NÃO USAR MICROSERVICES

MVP:

Modular

Monolith

Estrutura lógica:

```
Ether.Domain  
Ether.Application  
Ether.Infrastructure  
Ether.Contracts  
Ether.Api  
Ether.GameServer  
Ether.Worker
```

O sistema deve possuir boundaries suficientemente fortes para futura extração.

Não criar:

```
CombatService  
InventoryService  
QuestMicroservice  
EconomyMicroservice
```

neste momento.

4. SOLUTION

ProjectEther.sln

```
src/  
  Ether.Domain/  
  Ether.Application/  
  Ether.Infrastructure/  
  Ether.Contracts/  
  Ether.Api/  
  Ether.GameServer/  
  Ether.Worker/  
  
client/  
  Ether.Game/
```

```
tests/  
  Ether.Domain.Tests/  
  Ether.Application.Tests/  
  Ether.Infrastructure.Tests/  
  Ether.Api.Tests/  
  Ether.GameServer.Tests/  
  Ether.Architecture.Tests/
```

```
content/  
  classes/  
  creatures/  
  items/  
  abilities/  
  quests/  
  npcs/  
  maps/  
  recipes/  
  balance/
```

```
docs/  
  architecture/  
  protocol/  
  game-design/  
  operations/
```

5. DOMAIN PROJECT

Namespaces:

```
Ether.Domain.Common  
Ether.Domain.Accounts  
Ether.Domain.Characters  
Ether.Domain.Combat  
Ether.Domain.Creatures  
Ether.Domain.Items  
Ether.Domain.Inventory  
Ether.Domain.Quests  
Ether.Domain.Economy
```

```
Ether.Domain.Social
Ether.Domain.World
Ether.Domain.Crafting
```

6. DOMAIN PRIMITIVES

Criar:

```
AccountId
CharacterId
ItemInstanceId
ItemDefinitionId
CreatureDefinitionId
CreatureInstanceId
QuestId
GuildId
PartyId
MarketListingId
TradeId
AbilityId
MapId
```

Cada um deve encapsular Guid.

Não espalhar:

```
Guid                                characterId
Guid                                itemId
```

por todo o domínio quando o conceito possuir identidade própria.

7. MONEY

```
public      readonly      record      struct      Gold(long      Amount)
{
    public      static      Gold      Zero      =>      new(0);
```

```

public class Gold
{
    if (Amount < 0)
        throw new DomainException("Gold cannot be negative.");
}

```

Operações:

Add
Subtract
CanAfford

Não permitir:

negative	balance
overflow	
implicit	floating point

8. POSITION

```

public readonly record struct WorldPosition(
    MapId
    int X,
    int Y);

```

MVP não possui Z.

Adicionar Z posteriormente somente através de alteração de protocolo/versionamento.

9. CHARACTER AGGREGATE

```

Character
{

```

```
    CharacterId
    AccountId
    Name
    ClassId
    Level
    Experience
    Attributes
    Health
    Mana
    Position
    State
}
```

Métodos:

```
MoveTo
ApplyDamage
Heal
GrantExperience
LevelUp
Die
Respawn
Equip
Unequip
```

Nunca:

```
character.Level++
```

fora do domínio.

10. CHARACTER STATE

```
Offline
Loading
InWorld
Combat
Dead
```

Respawning
DisconnectGrace

Transições inválidas lançam:

InvalidCharacterStateException

11. CHARACTER ATTRIBUTES

MVP:

Strength
Dexterity
Intelligence
Vitality
Willpower

Cada atributo possui:

BaseValue
AllocatedValue
EquipmentModifier
BuffModifier
DebuffModifier
FinalValue

12. CLASS

MVP:

Warrior

Conteúdo futuro já previsto:

Ranger
Arcanist

Não implementar as duas classes futuras ainda.

13. WARRIOR

Base

High	survivability
Melee	
Shield	
Physical	damage

Abilities

Basic	Attack
Power	Strike
Shield	Guard
Dash	

14. ABILITY CONTRACT

```
AbilityDefinition
{
    AbilityId
    Name
    Type
    ResourceType
    ResourceCost
    Cooldown
    Range
    TargetRule
    DamageProfile
}
```

}

15. COMMAND BASE

Todos os comandos:

```
GameCommand
{
    CommandId
    SessionId
    CharacterId
    Sequence
}
```

CommandId:

UUID

Sequence:

monotonic

per

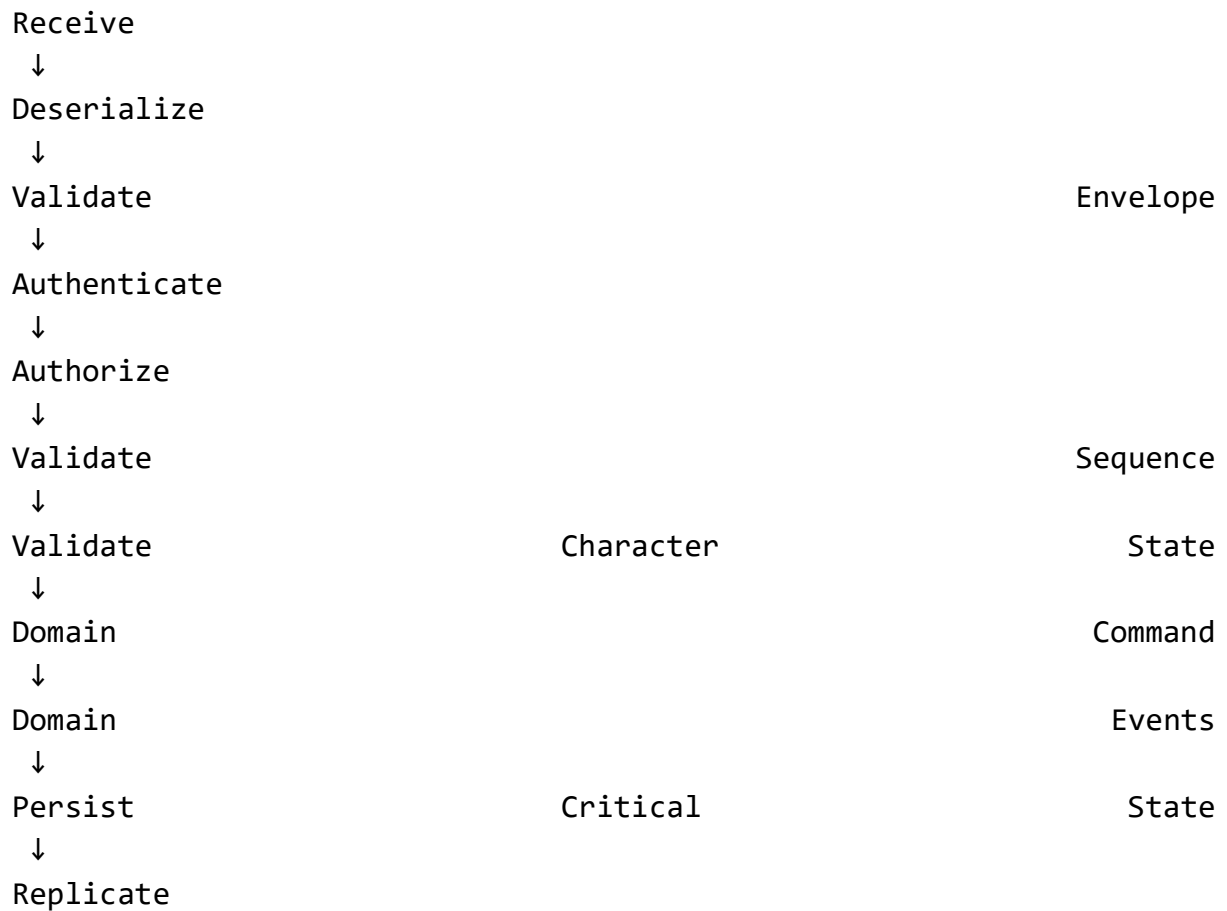
session

16. COMMANDS MVP

```
AuthenticateCommand
SelectCharacterCommand
MoveCommand
AttackCommand
CastAbilityCommand
InteractCommand
PickupCommand
EquipCommand
UnequipCommand
UseItemCommand
ChatCommand
```

CreatePartyCommand
InvitePartyCommand
LeavePartyCommand

17. COMMAND PIPELINE



18. COMMAND IDEMPOTENCY

Toda operação crítica deve suportar replay protection.

Exemplo:

CommandId = ABC

Se ABC chegar novamente:

DO

NOT

EXECUTE

AGAIN

19. MOVEMENT

Cliente envia:

```
{
  "type": "Move",
  "destination": {
    "x": 120,
    "y": 84
  }
}
```

Servidor calcula:

```
actual          movement
collision
speed
path
position
```

O cliente **nunca define sua posição final.**

20. MAP

Map

```
{
  MapId
  Width
  Height
  Tiles[]
}
```

Tile:

Walkable

BlocksSight

GroundType

SpeedModifier

DamageModifier

ZoneType

21. TILE

MVP:

Grass

Dirt

Stone

Water

Wall

Road

22. MOVEMENT VALIDATION

Servidor verifica:

CharacterAlive

CurrentMap

DestinationExists

DestinationWalkable

DistanceAllowed

MovementSpeed

Collision

23. CREATURE

```
CreatureInstance
{
    CreatureInstanceId
    CreatureDefinitionId
    Position
    Health
    State
    TargetId
}
```

24. CREATURE DEFINITIONS

MVP:

Slime
Wolf
Spider
Bandit
CaveBeast

25. CREATURE STATS

Cada criatura possui:

MaxHealth
Attack
Defense
MagicResistance
MovementSpeed
DetectionRange
AttackRange
AttackCooldown
ExperienceReward

LootTable

26. AI

Estados:

Idle
Patrol
Investigate
Chase
Attack
Flee
Return
Dead

27. AI LOOP

Não executar AI a cada frame do cliente.

Inicial:

5-10

Hz

Somente criaturas relevantes devem estar ativas.

28. INTEREST MANAGEMENT

Cada player recebe apenas entidades próximas.

InterestRadius

Fora desse raio:

no

replication

29. COMBAT

Ataque:

Player

↓

AttackCommand

↓

Target

Validation

↓

Range

↓

LOS

↓

Cooldown

↓

Damage

Calculation

↓

Damage

Applied

30. DAMAGE FORMULA

RawDamage

=

AbilityBase

+

WeaponPower

+

AttributeScaling

+

SkillScaling

Armor:

ArmorMultiplier

=

100 /

(100

+

EffectiveArmor)

Final:

```
FinalDamage =
max(
    1,
    RawDamage
    x ArmorMultiplier
    x ResistanceMultiplier
    x CriticalMultiplier
)
```

31. RANDOMNESS

Randomização:

```
SERVER ONLY
```

Nunca:

```
client RNG
client damage
client loot
```

32. DEATH

```
Health <= 0
↓
Die()
↓
CharacterState = Dead
↓
Death Event
↓
Resolve Penalty
↓
```

Respawn

Death penalty final continua:

PRODUCT_DECISION_REQUIRED

Portanto o agente deve implementar uma estratégia configurável, não uma regra irreversível.

33. LOOT

Loot é calculado pelo servidor.

CreatureDeath

↓

LootTable

↓

Roll

↓

GenerateItem

↓

Ownership

34. ITEM DEFINITION

ItemDefinition

{

Id

Name

Category

Stackable

MaxStack

EquipSlot?

BaseValue

Weight

```
}
```

35. ITEM INSTANCE

```
ItemInstance
{
    Id
    DefinitionId
    Quantity
    Durability?
    Owner
    Location
}
```

36. ITEM LOCATION

```
Inventory
Equipment
World
MarketEscrow
TradeEscrow
Crafting
Destroyed
```

Uma instância não pode existir em duas localizações simultaneamente.

37. INVENTORY

MVP:

30

slots

Stack máximo depende do item.

Exemplo:

Potion	=	20
Food	=	20
Material	=	100
Equipment	=	1

Valores são content data.

38. EQUIPMENT

Slots:

Weapon
Shield
Helmet
Chest
Legs
Boots

39. INVENTORY TRANSACTION

Toda movimentação:

Load
↓
Lock
↓
Validate
↓
Change
↓
Persist
↓
Audit

ownership/location

↓
Commit

40. XP

Implementação:

```
GrantExperience(  
    CharacterId,  
    long amount,  
    ExperienceSource source,  
    string referenceId)
```

Fontes:

CreatureKill
Quest
Discovery
Crafting
Event

41. XP CURVE

MVP:

$XP(level) = \text{floor}(100 \times level^{1.65})$

A fórmula deve ser encapsulada:

`IXperienceCurve`

para permitir balanceamento sem alterar domínio.

42. LEVEL

Level >= 1

Level up automático quando XP cruza threshold.

Evento:

CharacterLeveledUp

43. QUEST ENGINE

MVP objectives:

KillCreature
CollectItem
TalkToNpc
ReachLocation
UseAbility

44. QUEST STATE

NotStarted
Active
Completed
Failed
Abandoned

45. QUEST REWARD

Experience

Gold

Item

SkillExperience

Unlock

Entrega atômica.

46. NPC

MVP:

StarterGuide

Blacksmith

Merchant

QuestGiver

47. PARTY

MVP:

Create

Invite

Accept

Leave

Kick

Máximo:

5

players

Esse valor é configurável.

48. PARTY XP

Inicialmente:

Equal	eligible	split
-------	----------	-------

Somente jogadores elegíveis recebem XP.

A fórmula pode evoluir.

49. CHAT

MVP:

Local
Party
Private

World/Guild posteriormente.

50. WEBSOCKET

Endpoint:

wss://game.<domain>/game

ASP.NET Core possui suporte nativo a WebSockets e mecanismos de keep-alive/desconexão; isso é suficiente para o primeiro Game Server, sem adicionar uma camada de transporte desnecessária.

51. PROTOCOL ENVELOPE

```
{
  "version": 1,
  "type": "MoveCommand",
  "requestId": "uuid",
  "sequence": 100,
  "payload": {}
}
```

52. SERVER EVENT

```
{
  "version": 1,
  "type": "EntityMoved",
  "sequence": 101,
  "payload": {}
}
```

53. ERROR

```
{
  "version": 1,
  "type": "CommandRejected",
  "requestId": "uuid",
  "code": "OUT_OF_RANGE"
}
```

54. ERROR CODES

INVALID_COMMAND
INVALID_STATE
NOT_AUTHORIZED
NOT_FOUND
OUT_OF_RANGE
BLOCKED
COOLDOWN_ACTIVE
INSUFFICIENT_RESOURCE
INVALID_TARGET
INVENTORY_FULL
ITEM_NOT_OWNED
INVALID_DESTINATION
SESSION_EXPIRED
DUPLICATE_COMMAND

55. SESSION

```
GameSession
{
    SessionId
    AccountId
    CharacterId
    ConnectionId
    State
    ConnectedAt
    LastHeartbeatAt
}
```

56. RECONNECT

Socket	Lost
↓	
DisconnectGrace	

↓
Reconnect
↓
Authenticate
↓
ResumeSession
↓
Snapshot
↓
Delta
↓
InWorld

57. DISCONNECT GRACE

MVP:

30 seconds

Durante o grace period:

Character remains server-controlled

Em combate:

Character does NOT become invulnerable globally.

A política exata de proteção é:

PRODUCT_DECISION_REQUIRED

58. HEARTBEAT

Configuração inicial:

Ping:	20s
Timeout:	60s

O servidor encerra sessão considerada morta.

59. SERVER TICK

Inicial:

20	Hz
----	----

Não significa que todos os subsistemas executem 20 vezes por segundo.

Movement	20Hz
Combat	20Hz
AI	5Hz
Persistence	event/checkpoint
Replication	adaptive

60. GAME LOOP

Read	Commands
↓	
Validate	
↓	
Process	Commands
↓	
Simulation	
↓	
AI	
↓	
Combat	
↓	
Domain	Events
↓	

Replication

↓

Persistence

61. PERSISTENCE

Não salvar posição a cada tick.

Salvar em:

Login

Logout

MapChange

CriticalStateChange

InventoryChange

CombatRelevantCheckpoint

Death

LevelUp

QuestChange

EconomyTransaction

62. POSTGRESQL

Banco principal:

PostgreSQL

Utilizar:

transactions

constraints

foreign

indexes

row-level

keys

locking

Operações concorrentes críticas devem usar transações e locking apropriado; PostgreSQL suporta explicitamente FOR UPDATE e locks de linha para impedir alterações concorrentes no mesmo registro.

63. CORE TABLES

accounts
characters
character_attributes
character_skills
character_equipment
inventory_slots

item_definitions
item_instances

maps
map_tiles
spawn_definitions
creature_definitions

quest_definitions
quest_states

npc_definitions
dialogue_definitions

wallets
currency_transactions

parties
party_members

friends
blocks
reports

notifications
audit_events

64. ACCOUNTS

accounts

id	uuid	PK
external_identity		varchar
created_at		timestampz
updated_at		timestampz
status		varchar

65. CHARACTERS

characters

id	uuid	PK
account_id	uuid	FK
name		varchar(24)
class_id		varchar(32)
level		integer
experience		bigint
health		integer
mana		integer
map_id		uuid
position_x		integer
position_y		integer
state		varchar(32)
created_at		timestampz
updated_at		timestampz

Constraints:

level	>=	1
experience	>=	0
health	>=	0
mana	>=	0

Unique:

name

66. ITEM DEFINITIONS

item_definitions

id	uuid	PK
key	varchar(100)	UNIQUE
name		varchar(100)
category		varchar(50)
stackable		boolean
max_stack		integer
base_value		bigint
weight		integer
content_version		integer

67. ITEM INSTANCES

item_instances

id	uuid	PK
definition_id	uuid	FK
quantity		integer
durability	integer	NULL
location_type		varchar(32)
owner_character_id	uuid	NULL
created_at		timestampz
updated_at		timestampz

68. INVENTORY SLOTS

inventory_slots

character_id	uuid
slot_index	integer
item_instance_id	uuid NULL
PRIMARY	KEY(character_id, slot_index)

69. WALLET

wallets

character_id	uuid	PK
gold		bigint
premium_currency		bigint
version		bigint

Constraint:

gold	>=	0
premium_currency	>=	0

70. CURRENCY TRANSACTIONS

currency_transactions

id	uuid	PK
character_id		uuid
currency		varchar(32)
amount		bigint
direction		varchar(16)
balance_before		bigint

balance_after	bigint
source	varchar(64)
reference_id	varchar(128)
created_at	timestampz

71. AUDIT EVENTS

audit_events		

id	uuid	PK
actor_type		varchar(32)
actor_id	uuid	NULL
event_type		varchar(100)
aggregate_type		varchar(100)
aggregate_id	uuid	NULL
payload		jsonb
created_at		timestampz

72. EF CORE

DbContext:

EtherDbContext

Configurations:

```
AccountConfiguration
CharacterConfiguration
ItemDefinitionConfiguration
ItemInstanceConfiguration
InventorySlotConfiguration
WalletConfiguration
...
```

Nunca colocar configuração pesada dentro das entidades.

73. EF CORE MIGRATIONS

Diretório:

Ether.Infrastructure/Migrations

Toda mudança:

Migration

↓

Integration

↓

Database

Upgrade

Tests

Test

Nunca modificar migration já aplicada.

74. REPOSITORIES

Interfaces no Application/Domain boundary:

ICharacterRepository

IItemRepository

IInventoryRepository

IQuestRepository

IWalletRepository

Infrastructure implementa.

75. UNIT OF WORK

IUnitOfWork

Usado em operações transacionais.

Mas não criar UnitOfWork artificial em toda chamada.

Transaction boundary deve seguir caso de uso.

76. APPLICATION COMMANDS

Exemplo:

```
MoveCharacterCommand
AttackTargetCommand
CastAbilityCommand
PickupItemCommand
EquipItemCommand
GrantExperienceCommand
CompleteQuestCommand
```

Handlers:

```
MoveCharacterHandler
AttackTargetHandler
...
```

77. DOMAIN EVENTS

MVP:

```
CharacterCreated
CharacterMoved
CombatStarted
DamageApplied
CreatureKilled
LootGenerated
ItemTransferred
ItemEquipped
ExperienceGranted
```

CharacterLeveledUp
CharacterDied
CharacterRespawned
QuestStarted
QuestCompleted
PartyCreated
PartyMemberJoined

78. EVENT RULE

Domain Event representa fato ocorrido.

Não utilizar evento como comando.

Errado:

MoveCharacterEvent

se o significado é uma solicitação.

Correto:

MoveCharacterCommand

e posteriormente:

CharacterMoved

79. OUTBOX

Eventos que precisam sair do processo:

Domain

↓

Outbox

↓

Event

Worker

↓

Redis

/

Analytics

/

Notifications

Não publicar diretamente em infraestrutura externa dentro do transaction critical path.

80. OUTBOX TABLE

outbox_messages

id	uuid	PK
type		varchar(200)
payload		jsonb
occurred_at		timestampz
processed_at	timestampz	NULL
attempts		integer
last_error	text	NULL

81. REDIS

Uso inicial:

session	coordination
presence	
distributed	coordination
transient	state
rate	limiting

Não usar Redis como fonte definitiva de:

gold	
inventory	
character	progression
ownership	

82. RATE LIMITING

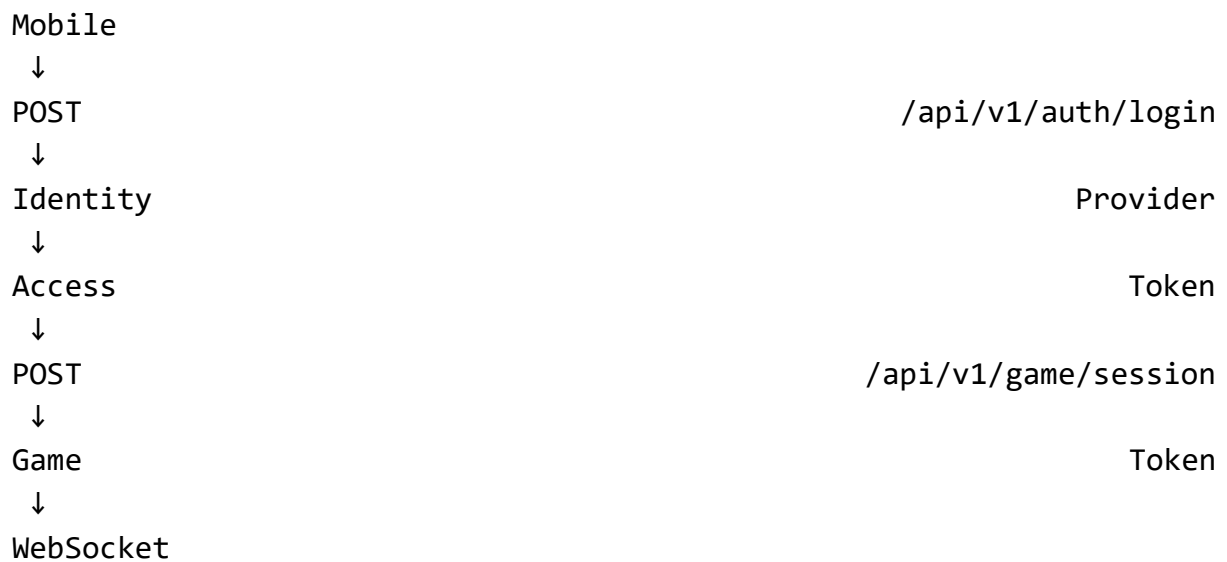
Limites separados:

Login
Chat
Movement
Attack
Ability
Trade
Market
Reconnect

Nunca usar um único rate limiter global.

83. AUTH

Fluxo:



84. GAME TOKEN

Token específico do Game Server.

Claims:

AccountId
CharacterId
SessionId
Expiration

85. AUTHORIZATION

Antes de qualquer comando:

Token				valid?
Session				valid?
Character	belongs		to	session?
Character		state		valid?

86. CLIENT STRUCTURE

Godot:

client/Ether.Game/

scenes/
 bootstrap/
 login/
 character/
 world/
 ui/

scripts/
 networking/
 world/

```
combat/  
inventory/  
quests/  
ui/  
input/  
  
assets/  
  characters/  
  creatures/  
  items/  
  maps/  
  effects/  
  audio/
```

87. CLIENT ARCHITECTURE



Não permitir que UI altere diretamente authoritative state.

88. CLIENT INPUT

Mobile:



89. ONE-HAND MODE

Com uma mão:

Movement

Basic

Target

Interact

Potion

Attack

devem ser acessíveis.

90. TWO-HAND MODE

Duas mãos permitem:

Joystick

Ability

Target

Camera

buttons
management

simultaneamente.

91. CAMERA

MVP:

Fixed

top-down

com:

small

camera

smoothing

Não implementar câmera livre inicialmente.

92. RENDERING

Objetivo:

2D

HD

stylized

Não:

tiny

pixel

art

93. SERVER SNAPSHOT

Ao entrar:

```
{
  "character": {},
  "position": {},
  "health": {},
  "mana": {},
  "inventory": [],
  "equipment": [],
  "quests": [],
  "entities": []
}
```

94. REPLICATION

Mensagens:

EntitySpawned

EntityUpdated

EntityMoved
EntityDamaged
EntityDied
EntityDespawned

95. DELTA COMPRESSION

Somente mudanças são enviadas depois do snapshot inicial.

Exemplo:

HP:
100 → 82

Enviar:

82

não toda a entidade.

96. CONTENT PIPELINE

Conteúdo será data-driven.

Exemplo:

```
{  
  "key": "warrior_power_strike",  
  "baseDamage": 25,  
  "cooldownMs": 3000,  
  "range": 1  
}
```

97. CONTENT VALIDATION

CI deve falhar se:

duplicate	ID
missing	reference
invalid	probability
negative	value
invalid	range
invalid	quest
invalid	recipe
invalid	item

98. LOOT TABLE

Formato:

```
{
  "key": "wolf_loot",
  "entries": [
    {
      "item": "wolf_fang",
      "chance": 0.35,
      "min": 1,
      "max": 2
    }
  ]
}
```

99. QUEST DATA

```
{
  "key": "first_hunt",
  "objectives": [
    {
```

```

        "type": "KillCreature",
        "target": "wolf",
        "quantity": 3
    }
],
"rewards": {
    "experience": 100,
    "gold": 25
}
}

```

100. ITEM SEED

Initial:

```

wooden_sword
iron_sword
wooden_shield
leather_helmet
leather_chest
leather_legs
leather_boots
health_potion
bread
wolf_fang
slime_core
spider_silk
bandit_token

```

101. CREATURE SEED

```

slime
wolf
spider
bandit

```

cave_beast

102. MAP SEED

starter_town
green_fields
whispering_forest
old_cave
forgotten_ruins

103. QUEST SEED

Primeiras quests:

welcome_to_ether
first_hunt
gathering_supplies
into_the_forest
the_old_cave

104. NPC SEED

starter_guide
blacksmith
merchant
hunter
elder

105. INITIAL WORLD



Pequeno.

Denso.

Explorável.

106. FIRST 30 MINUTES

O jogador deve experimentar:

Character	creation
Movement	
NPC	
First	quest
First	creature
First	combat
First	loot
First	equipment
Level	up
New	ability
Exploration	
Second	creature
Optional	path

107. FIRST 5 MINUTES

Objetivo:

movement
combat
loot
reward

O jogador precisa entender rapidamente:

“O que faço aqui?”

108. FIRST 30 MINUTES — NÃO FAZER

Não obrigar:

tutorial		longo
cutscene		extensa
20		telas
15		menus
daily		reward
energy		system
forced	store	interaction
forced		guild
forced		PvP

109. TEST STRATEGY

Unit

Domínio puro.

Integration

PostgreSQL real.

Contract

Protocol JSON.

GameServer

Simulation.

E2E

Client ↔ Server.

110. TEST COVERAGE

Não perseguir porcentagem artificial.

Prioridade:

100%	critical	invariants
100%	economy	operations
100%	ownership	transitions
100%	combat	formulas
100%	security	boundaries

111. CRITICAL TEST

Given	player	has	100	gold
And	listing	costs	100	
When		purchase		succeeds
Then	player	has	0	gold
And	item	belongs	to	player
And	seller	receives		proceeds

And listing is closed
And exactly one transaction exists

112. DUPLICATION TEST

Executar:

PurchaseCommand
PurchaseCommand

com mesmo:

CommandId

Resultado:

one purchase

113. CONCURRENCY TEST

Dois players comprando último item:

Player	A	→	Purchase
Player	B	→	Purchase

Somente um consegue comprar.

PostgreSQL transaction/locking deve garantir isso.

114. INVENTORY DUPLICATION TEST

Executar simultaneamente:

Equip
Trade
Destroy

sobre o mesmo item.

Resultado:

exactly one valid state

Nunca:

duplicated item

115. COMBAT SECURITY TEST

Cliente envia:

```
{  
  "damage": 999999  
}
```

Servidor ignora o campo inexistente/não autorizado.

Dano é recalculado.

116. MOVEMENT SECURITY TEST

Cliente envia:

```
{  
  "x": 999999,  
  "y": 999999  
}
```

Servidor rejeita.

117. COOLDOWN SECURITY TEST

Cliente dispara ability antes do cooldown.

Resultado:

COOLDOWN_ACTIVE

118. LOOT SECURITY TEST

Cliente tenta:

PickupItem(otherPlayerItem)

Resultado:

NOT_AUTHORIZED

119. RECONNECT TEST

Login

↓

Spawn

↓

Kill

creature

↓

Receive

loot

↓

Disconnect

↓

Reconnect

Resultado:

loot	exists	exactly	once
XP	exists	exactly	once
character		position	valid

120. SERVER CRASH TEST

Durante operação não crítica:

Server	crash
↓	
Restart	
↓	
Reconnect	

State persistent deve permanecer consistente.

121. ADMIN

MVP admin functions:

- FindPlayer
- InspectCharacter
- InspectInventory
- InspectWallet
- InspectTransactions
- InspectAudit
- Ban
- Mute
- GrantItem
- GrantGold
- Teleport

ResetCharacter

Toda operação administrativa gera audit.

122. ADMIN GRANT

Não executar:

```
UPDATE wallets SET gold = gold + 1000;
```

manual.

Executar:

AdminGrantGoldCommand

para manter:

```
audit  
transaction  
validation
```

123. OBSERVABILITY

Metrics:

```
connected_players  
active_sessions  
commands_per_second  
commands_rejected  
combat_per_second  
database_latency  
tick_duration  
tick_overruns  
websocket_disconnects
```

reconnect_success

124. GAME METRICS

characters_created
first_move
first_attack
first_kill
first_loot
first_level_up
first_quest
first_death
session_duration
return_session

125. BUSINESS METRICS

Posteriormente:

D1
D7
D30
conversion
ARPU
payer_conversion
retention

Não otimizar monetização antes de validar core gameplay.

126. LOGGING

Logs estruturados.

Exemplo:

```
{
  "event": "combat.damage_applied",
  "characterId": "...",
  "targetId": "...",
  "damage": 42,
  "formulaVersion": 1
}
```

127. CORRELATION

Toda request:

CorrelationId

Toda sessão:

SessionId

Toda operação crítica:

CommandId

128. DOCKER COMPOSE

Serviços iniciais:

```
postgres
redis
ether-api
ether-game-server
ether-worker
```

Development:

docker

compose

up

-d

129. ENVIRONMENT VARIABLES

ConnectionStrings__Postgres
Redis__ConnectionString
Jwt__Issuer
Jwt__Audience
Jwt__SigningKey
GameServer__Endpoint
OpenTelemetry__Endpoint

Secrets fora do repository.

130. CI

Pipeline:

restore
build
unit-tests
integration-tests
architecture-tests
content-validation
docker-build

Merge bloqueado se falhar.

131. ARCHITECTURE TESTS

Proibir:

Domain	→	Infrastructure
Domain	→	EF
Domain	→	ASP.NET
Domain	→	Godot

132. FIRST 20 TASKS

ETHER-001	Create	solution
ETHER-002	Configure	projects
ETHER-003	Configure	architecture tests
ETHER-004	Domain	primitives
ETHER-005	Character	aggregate
ETHER-006	Account	persistence
ETHER-007	Character	persistence
ETHER-008		Authentication
ETHER-009	Game	session
ETHER-010	WebSocket	endpoint
ETHER-011	Protocol	envelope
ETHER-012	Map	model
ETHER-013		Movement
ETHER-014	Creature	model
ETHER-015	Creature	spawning
ETHER-016	Combat	engine
ETHER-017	Damage	formulas
ETHER-018		Death
ETHER-019		Loot
ETHER-020		Inventory

133. TASKS 21–40

ETHER-021	Equipment
ETHER-022	XP
ETHER-023	Level
ETHER-024	Abilities
ETHER-025	Cooldowns
ETHER-026	Quests

ETHER-027		NPC
ETHER-028		Dialogue
ETHER-029		Party
ETHER-030		Chat
ETHER-031		Reconnect
ETHER-032		Snapshot
ETHER-033		Replication
ETHER-034	Interest	management
ETHER-035		Outbox
ETHER-036		Analytics
ETHER-037		Audit
ETHER-038		Admin
ETHER-039	Content	validator
ETHER-040	Seed	data

134. TASKS 41–60

ETHER-041	Godot	bootstrap
ETHER-042	Godot	login
ETHER-043	Godot	websocket
ETHER-044	Godot	protocol
ETHER-045	Character	rendering
ETHER-046		Camera
ETHER-047	Touch	movement
ETHER-048	Virtual	joystick
ETHER-049	Target	selection
ETHER-050	Combat	UI
ETHER-051	Ability	bar
ETHER-052	Health	UI
ETHER-053	Inventory	UI
ETHER-054	Equipment	UI
ETHER-055	Quest	UI
ETHER-056	NPC	dialogue
ETHER-057	Chat	UI
ETHER-058	Party	UI
ETHER-059	Reconnect	UI
ETHER-060	Mobile	performance
		pass

135. TASKS 61–80

ETHER-061	PostgreSQL	integration	tests
ETHER-062		Transaction	tests
ETHER-063		Concurrency	tests
ETHER-064		Duplication	tests
ETHER-065		Security	tests
ETHER-066	Movement	exploit	tests
ETHER-067	Combat	exploit	tests
ETHER-068	Inventory	exploit	tests
ETHER-069		Economy	tests
ETHER-070		Load	tests
ETHER-071		Tick	profiling
ETHER-072		Network	profiling
ETHER-073		Database	profiling
ETHER-074		Redis	integration
ETHER-075		Rate	limiting
ETHER-076		Anti-bot	telemetry
ETHER-077			Backup
ETHER-078			Restore
ETHER-079		Disaster	recovery
ETHER-080		Production	deployment

136. TASKS 81–100

ETHER-081		Tutorial
ETHER-082	Starter	Town
ETHER-083		Fields
ETHER-084		Forest
ETHER-085		Cave
ETHER-086		Ruins
ETHER-087	Creature	balancing
ETHER-088	Item	balancing
ETHER-089	Combat	balancing
ETHER-090	XP	balancing
ETHER-091	Quest	balancing

ETHER-092		Mobile	UX	pass
ETHER-093			Audio	pass
ETHER-094			VFX	pass
ETHER-095		First	closed	playtest
ETHER-096			Telemetry	analysis
ETHER-097	Fix	critical	gameplay	problems
ETHER-098			Second	playtest
ETHER-099			Release	candidate
ETHER-100	MVP	vertical	slice	sign-off

137. DEFINITION OF DONE

Uma task só está DONE quando:

Implementation

+

Tests

+

Validation

+

No known critical regression

+

Documentation when applicable

138. TASK REPORT

Todo agente deve finalizar informando:

TASK:

STATUS:

CHANGED:

- ...

TESTS:

- ...

MIGRATIONS:

-

...

SECURITY:

-

...

PERFORMANCE:

-

...

SPEC_GAP:

-

none

SPEC_CONFLICT:

-

none

NEXT:

-

...

139. AGENT PROMPT

O agente de desenvolvimento deve receber:

Você está implementando o Project Ether.

O Blueprint v4.0 é contrato.

Não invente regras de negócio.

Não simplifique invariantes.

Não confie no cliente.

Não remova testes existentes.

Não altere comportamento fora do escopo.

Se encontrar ambiguidade crítica, reporte SPEC_GAP.

Se encontrar conflito entre regras, reporte SPEC_CONFLICT.

Se precisar de uma decisão de produto, reporte PRODUCT_DECISION_REQUIRED.

Toda alteração deve possuir testes.

Toda alteração de banco deve possuir migration.

Toda operação econômica deve ser atômica.

Toda alteração de ownership deve ser auditável.

Toda fórmula deve ser determinística e versionável.

O servidor é authoritative.

O cliente é não confiável.

Implemente incrementalmente.

Antes de alterar arquitetura, prove que a arquitetura atual é insuficiente.

140. PRIMEIRA TASK DO AGENTE

ETHER-001

Objetivo

Criar a solution inicial.

Criar

ProjectEther.sln

Ether.Domain

Ether.Application

Ether.Infrastructure

Ether.Contracts

Ether.Api

Ether.GameServer

Ether.Worker

Ether.Domain.Tests

Ether.Application.Tests

Ether.Infrastructure.Tests

Ether.Api.Tests

Ether.GameServer.Tests
Ether.Architecture.Tests

Regras

Target:

net10.0

Nullable:

enable

Implicit usings:

enable

Warnings:

TreatWarningsAsErrors

141. ETHER-001 ACCEPTANCE

dotnet

restore

passa.

dotnet

build

passa.

dotnet

test

passa.

Architecture tests passam.

Nenhum projeto possui dependência proibida.

142. ETHER-002

Criar:

AccountId
CharacterId
ItemInstanceId
ItemDefinitionId
CreatureInstanceId
CreatureDefinitionId
QuestId
AbilityId
MapId
PartyId
GuildId
MarketListingId
TradeId

Todos:

readonly	record	struct
----------	--------	--------

143. ETHER-002 ACCEPTANCE

Testar:

Equality		
Empty/default		behavior
Serialization	where	needed

IDs não podem ser implicitamente confundidos.

144. ETHER-003

Criar:

DomainException
InvalidStateException
InvariantViolationException
ConcurrencyException

145. ETHER-004

Implementar:

Character
CharacterState
CharacterAttributes
WorldPosition

com invariantes.

146. ETHER-005

Criar testes do Character.

Obrigatório:

Cannot	create	level	0
Cannot	create	negative	XP
Cannot	damage	dead	character
Cannot	move	dead	character
Cannot	heal	beyond	max
Cannot		level	down

147. ETHER-006

Criar PostgreSQL development environment.

Docker:

```
postgres:18
redis:latest
```

Versões devem ser fixadas no repository quando o ambiente for estabilizado.

148. ETHER-007

Criar EF Core.

Implementar:

```
Account
Character
CharacterAttributes
```

149. ETHER-008

Migration inicial.

Executar:

```
dotnet ef migrations add InitialSchema
dotnet ef database update
```

somente em development.

Production utiliza migration controlada.

150. ETHER-009

Criar API:

GET	/health/live
GET	/health/ready

151. ETHER-010

Criar WebSocket:

/game

Com:

connection
authentication
heartbeat
disconnect

152. ETHER-011

Implementar envelope:

GameMessage
GameCommand
GameEvent
CommandRejected

153. ETHER-012

Implementar:

Login
Character
Session

selection
creation

154. ETHER-013

Implementar mapa mínimo:

32 × 32

com:

walkable
blocked

155. ETHER-014

Implementar movimento.

Primeiro somente:

A → B

sem criaturas.

156. ETHER-015

Implementar segundo player.

Teste:

Player		A		moves
Player	B		sees	movement

157. ETHER-016

Implementar criatura.

Primeira:

Slime

158. ETHER-017

Implementar:

Basic

Attack

159. ETHER-018

Implementar:

Damage

Death

Respawn

160. ETHER-019

Implementar loot:

Slime

Core

161. ETHER-020

Implementar inventory.

Resultado:

Player

↓

Kill

Slime

↓

Loot

Slime

Core

↓

Inventory

162. FIRST VERTICAL SLICE

A primeira milestone real deve ser:

Open

game

↓

Login

↓

Create

Warrior

↓

Enter

Town

↓

Walk

↓

Leave

Town

↓

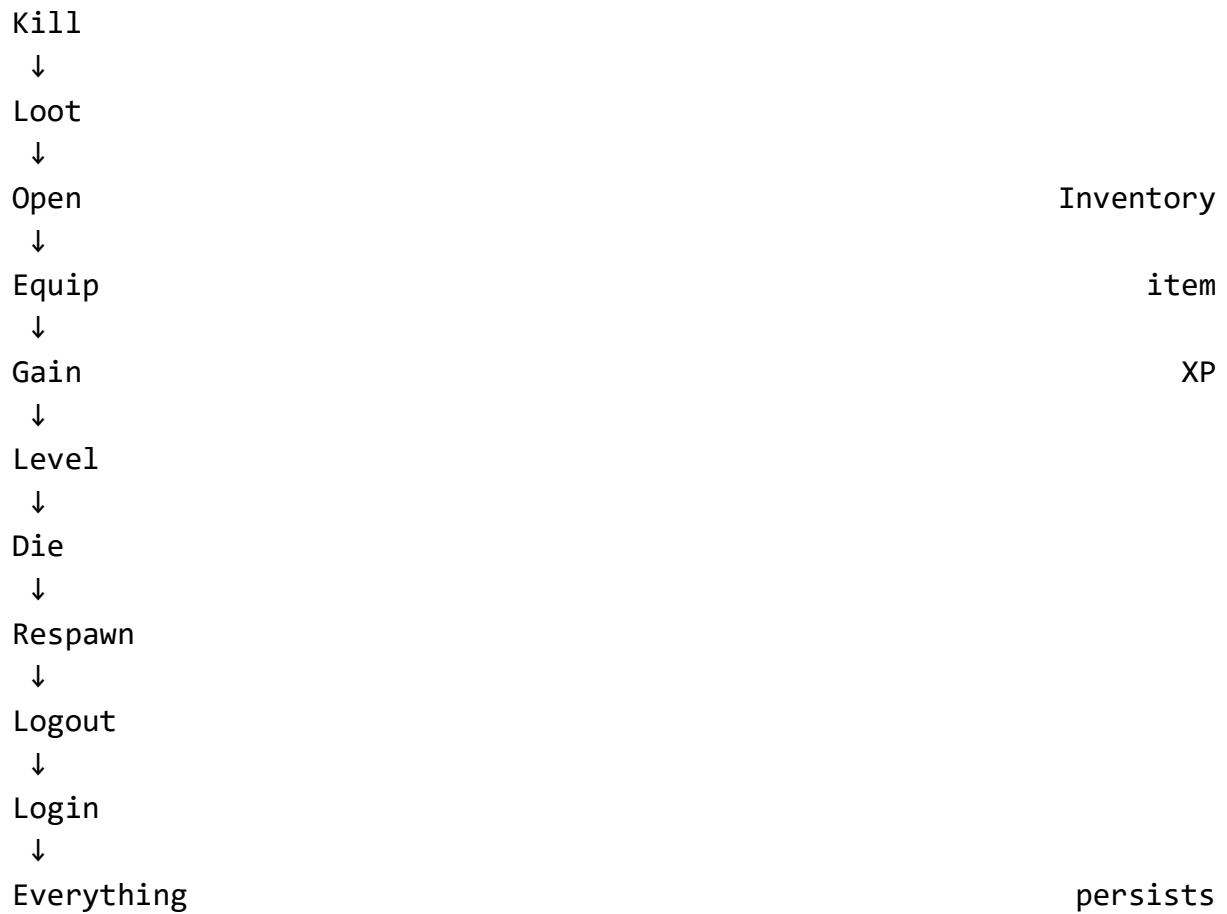
Find

Slime

↓

Attack

↓



163. SECOND VERTICAL SLICE

Adicionar:



164. THIRD VERTICAL SLICE

Adicionar:

NPC



Quest



Objective



Reward



XP



Item

165. FOURTH VERTICAL SLICE

Adicionar:

Crafting



Item



Trade



Gold

166. FIFTH VERTICAL SLICE

Adicionar:

Market



Listing



167. NÃO IMPLEMENTAR ANTES

Não implementar:

guild	wars
housing	
mounts	
pets	
open	PvP
premium	shop
battle	pass
gacha	
world	bosses
large-scale	events

antes de validar os cinco vertical slices.

168. GAMEPLAY QUALITY GATE

Depois do quinto slice:

parar desenvolvimento de features.

Executar:

playtest
telemetry
performance
security

economy

simulation

169. PLAYTEST QUESTIONS

Medir:

Where		did		players		stop?
Where		did		players		get
How		long		until		first
How		long		until		first
How	often		did	they		move
How		often		did		they
How		often		did		they

170. NÃO USAR MÉTRICA COMO SUBSTITUTO DE DESIGN

Analytics serve para informar decisões.

Não significa:

metric				changed
→		automatically		change
				game

171. ECONOMY SIMULATION

Antes de marketplace:

similar:

100
1,000
10,000

100,000

characters.

Medir:

gold
gold
item
item
wealth

creation
destruction
creation
destruction
concentration

172. INFLATION CONTROL

Sources:

Monster
Quest
NPC

gold
rewards
sales

Sinks:

NPC
repair
market
crafting
teleports
future

purchases

tax
fees

services

Valores serão balanceados após simulação.

173. ITEM ECONOMY

Cada item relevante terá:

CreationRate
DestructionRate
Circulation
AveragePrice
MedianPrice

174. PLAYER MARKET

Não permitir que o marketplace seja apenas:

NPC shop com outra interface.

Ele deve criar:

price
specialization
trade
scarcity

discovery

175. SOCIAL DESIGN

Não obrigar guild para progredir.

Mas criar atividades em que:

cooperation

produza vantagens reais.

176. MOBILE DESIGN RULE

Não transformar o MMORPG em:

auto
idle
energy

battler
game
game

O jogo continua sendo jogo.

O mobile adapta:

input
session
UI
reconnect
notifications

length

não a essência.

177. MONETIZATION RULE

Inicialmente:

NO
NO
NO
NO

POWER

FORCED

PAY-TO-WIN
GACHA
ENERGY
ADS

Monetização futura:

Cosmetics
Optional
Optional
Expansion
Convenience

Pass
Subscription
Content

Cada uma será especificada posteriormente.

178. CONTENT PRINCIPLE

Preferir:		
10	criaturas	memoráveis
a:		
100	criaturas	genéricas.
Preferir:		
10	quests	interessantes
a:		
300	fetch	quests.

179. WORLD PRINCIPLE

O mundo não deve ser um corredor de:	
Quest	marker
↓	
Combat	marker
↓	
Reward	marker
↓	
Next	marker

Exploração deve existir.

180. CORE DIFFERENTIATOR

Project Ether não deve competir apenas por:

graphics		
content		quantity
number	of	classes

Seu diferencial deve surgir da combinação:

Persistent		World
+		
Exploration		
+		
Player		Economy
+		
Meaningful		Progression
+		
Social		Interaction
+		
Mobile		Accessibility

181. FINAL ARCHITECTURAL RULE

Se o sistema funcionar com:

1,000	players
-------	---------

mas precisar ser reescrito para:

10,000

isso é aceitável no MVP.

Se ele não funcionar corretamente com:

10	players
----	---------

a arquitetura está errada.

182. FINAL DEVELOPMENT RULE

Nunca construir:

“o MMORPG inteiro”.

Construir:

uma experiência pequena

que funcione.

Depois:

melhorar

medir

expandir

183. PRIMEIRO OBJETIVO REAL

Ao final do primeiro ciclo:

Player	1
--------	---

Player	2
--------	---

↓

Same World

↓

Combat

↓

Loot

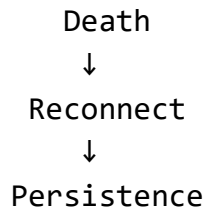
↓

Progression

↓

Quest

↓



Se isso for divertido:

temos um jogo.

Se não for:

não adianta adicionar 500 sistemas.

184. ESTADO DO PROJETO

Após v4.0:

PRODUCT	DESIGN			READY
DOMAIN	MODEL			READY
ARCHITECTURE				READY
PROTOCOL				READY
DATABASE	STRATEGY			READY
SECURITY	MODEL			READY
TEST	STRATEGY			READY
MVP	CONTENT			DEFINED
IMPLEMENTATION		READY	TO	START
BALANCE			PARTIALLY	OPEN
LORE				OPEN
FINAL	ART			OPEN
MONETIZATION				OPEN

185. PRÓXIMA AÇÃO

A partir daqui, o trabalho deixa de ser:

"pensar" no "jogo"

e passa a ser:

IMPLEMENT

ETHER-001

Depois:

ETHER-002

ETHER-003

...

Cada milestone deve produzir algo jogável e testável.

Não avançar para a próxima camada enquanto a anterior não estiver funcional.

186. DEFINIÇÃO FINAL DO MVP

O MVP está completo quando:

Authentication	✓
Character	✓
World	✓
Movement	✓
Multiplayer	✓
Combat	✓
Creature AI	✓
Loot	✓
Inventory	✓
Equipment	✓
XP	✓
Level	✓
Quest	✓
NPC	✓
Death	✓
Respawn	✓

Reconnect	✓
Persistence	✓
Party	✓
Chat	✓
Crafting	✓
Trade	✓
Basic Economy	✓
Marketplace	✓
Admin	✓
Audit	✓
Telemetry	✓
Anti-duplication	✓
Mobile Client	✓

E principalmente:

Players actually want to play again.

Esse último requisito não pode ser substituído por nenhuma quantidade de código, testes ou features.